Computer Programming

Compiled by H Msukwa (0881187149)

Computer Programming

- The unit aims at equipping leaners with various techniques and frameworks , used in creating base computer programs
- Techniques used in creating computer programs (coupling , cohesion , trace tables , flow chart etc.)
- Frameworks used in creating computer programs(object oriented programming, functional programming , imperative programming etc)

Computer Programming

- This unit undertakes the sub topic
  - Evaluate common programming languages
  - Apply common data types, data structures, and algorithms to problem solving
  - Analyse the process of source code compilation and machine code execution
  - Apply code tracing methods to determine the output of a function for a given input
  - Analyse the advantages of object-orientated programming

Computer Programming

Computer Program

- A set or list of logical instructions , coded in a particular programming language that attains certain tasks or functions
- Computer programs are of various types namely:

- Air traffic system
- Flood control system
- Inventory management system
- School management system
- Sales management system

Evaluate common programming languages

Programming Language

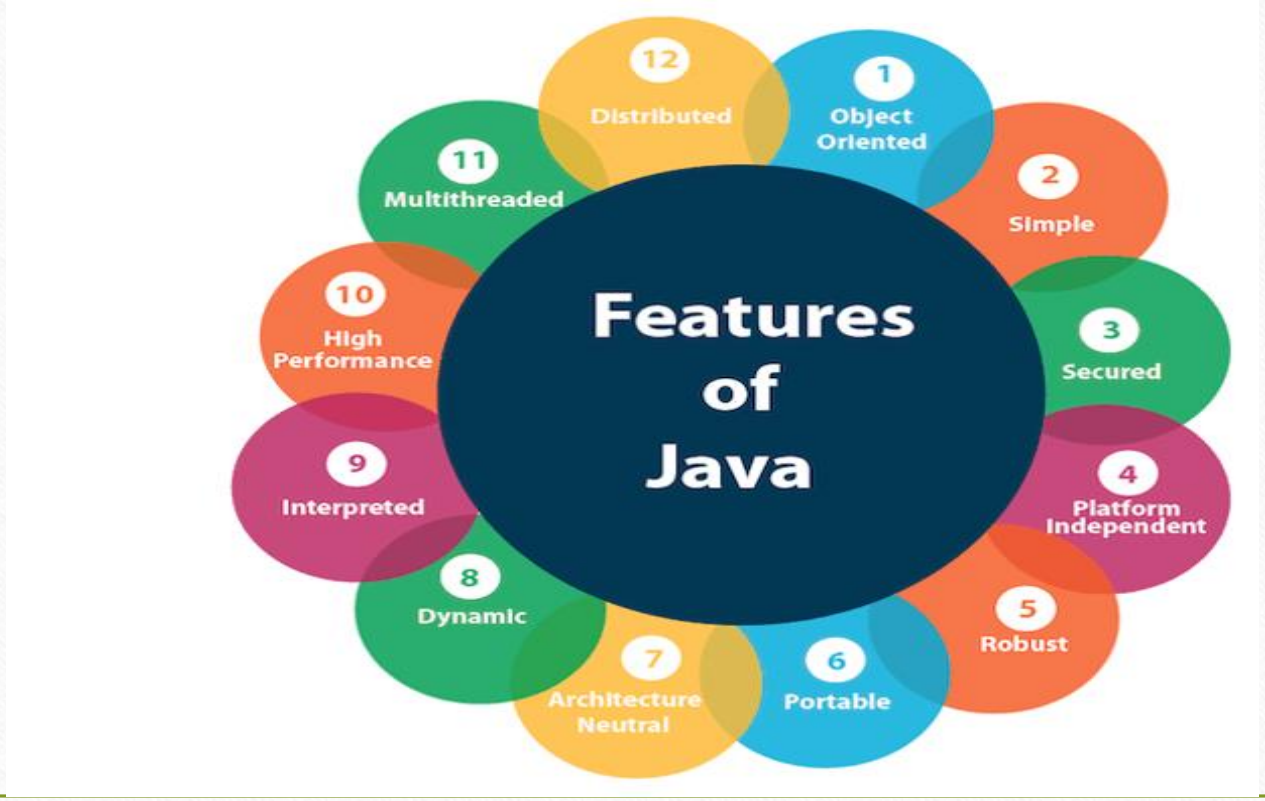
- It's a language used in creating or coding computer program
- It communicates instructions to computer or machine
- Common types of programming languages are: C# , JAVA , Ruby, C, C++, machine code , Assembly language etc.
- Programming languages are categorized into high level languages and low level languages and formed into interpreted and compiled languages

Common or types of Programming Languages

1. Java Language

- A multi-purpose computer programming language which is a class based and object oriented
- It is a write once and run anywhere language meaning it runs on any application that supports java
- The language was developed by James Gosling in 1995 at sun microsystems

Features of Java Language



Features of Java Language

1. **Simple**

- Java syntax is easy to understand and has removed some complex features such as explicit pointers , operator overriding

2. **Object Oriented language**

- Involves organising the software as a combination of different types of objects that in cooperates both data and behaviour
- Provides a proper framework for implementing the software i.e. in classes

Features of Java Language

3. **Platform Independent (platform is the hardware architecture (software architecture) of machine where the program will be installed)**

It's a write once and run any where language hence its byte code can be run on multiple platforms i.e. windows , Linux ,sun Solaris, Mac /OS , Intel processor , macro processor etc.

- It does not need to be compiled into specific machines
- There are two types of platforms namely: software based and hardware based hence java provides a software based platform

Features of Java Language

4. Secure

- Java provides a robust code since it comprises of:
- Byte code verifier- it checks the code fragments for illegal code that can violate access right to objects i.e. exceptional handler
- Security manager – it determines what resources a class can access such as reading and writing to the local disk

Features of Java Language

5. Portable

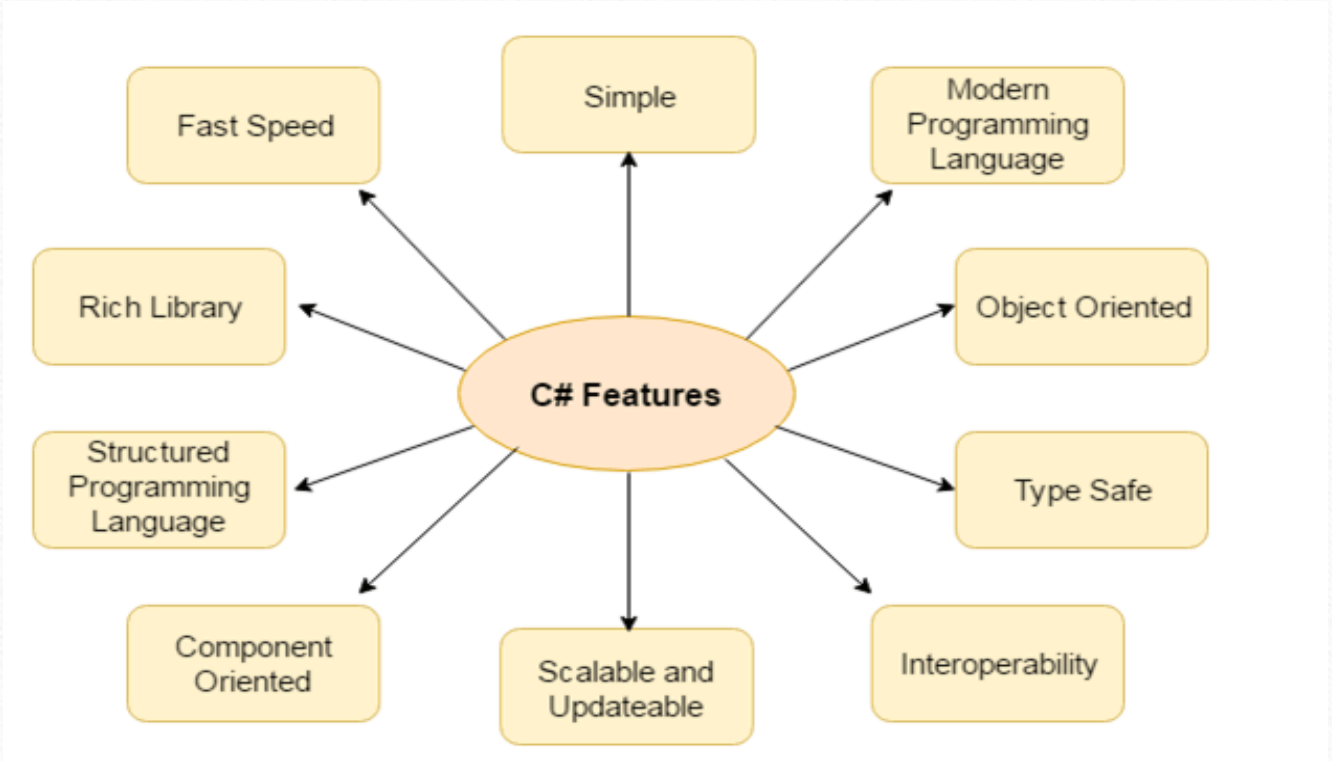
- Java byte code are portable as they could be carried on any platform and does not require any implementation

Uses of Java Language
❑ Used in creating mobile applications ❑ Used in creating server applets ❑ Used in creating / connecting web applications

C# language

- An object oriented programming language for Microsoft .NET platforms
- Associated with automatic memory management hence frees unused variable space
- It's a drag and drop language hence program developing is fast

Features of C# language



Features of C# language

1. Simple

- It is simple to create a program as it involves dragging and dropping features on the window

2. Object Oriented

- It involves the use of object oriented construct such as classes , object , polymorphism , inheritance etc.

Features of C# language

3. Robust

- It comprises of strong memory management that automatically freezes unused space for variable declaration

4. Rich Library

- Provides a lot of inbuilt functions that makes the development fast

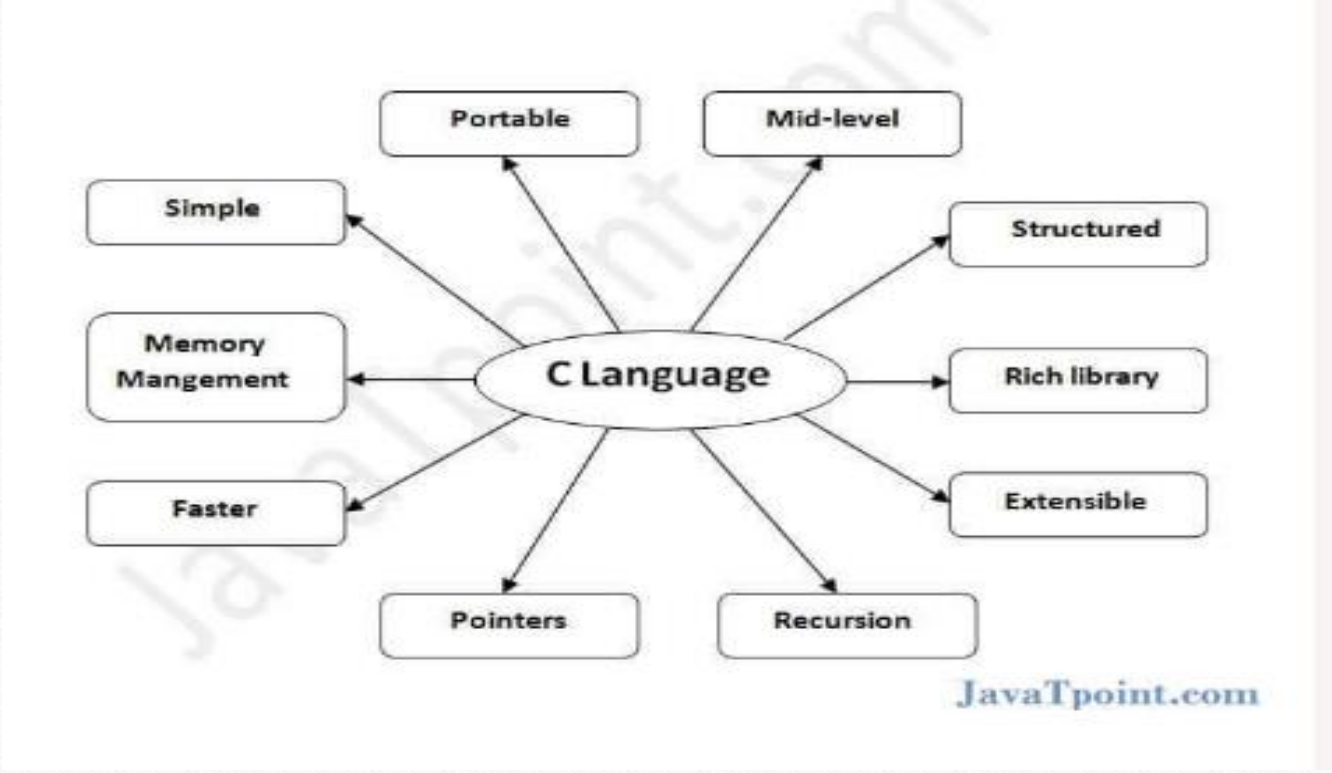
5 Fast Speed

- It has got the fast or quick program compilation and execution time

C Language

- A compiled based language that support imperative and structured programming styles
- Used for system programming including operating systems & embedded systems since it is a light weight language
- The language was developed in 1972 by Dennis Ritchie & was developed to be used in UNIX operating system to overcome the problems of the past languages namely: B ,BCPL

Features of C language



Features of C language

1 Simple

- Provides a structured approach to break the problem or program into parts

2 Machine independent or portable

- The program can be executed on different machines

3 Provides a structured style

- Helps to break the program into parts using functions hence it is easily to understand & modify the program
- Functions provides code reusability

Features of C language

5 .Memory management

- Enhances a dynamic memory allocation hence allows to free the allocated memory at any time by calling Free() function

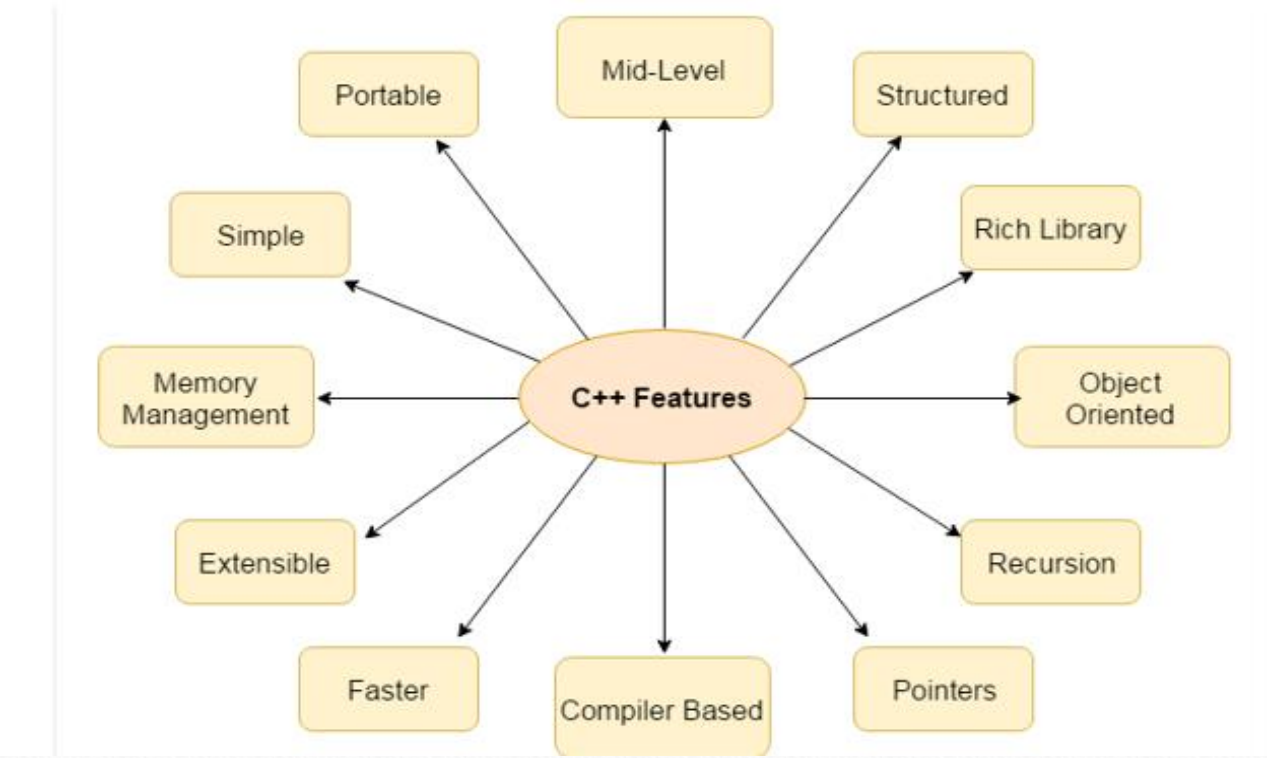
6 .Pointer usage

- Helps to quickly interact with the system memory , data structures , arrays, functions etc.

C++ Language

- A compiled based or middle-level language that encapsulates both high and low level languages features
- It is a case sensitive language that supports object oriented , procedural and generic programming style

Features of C ++



Features of C++ Language

1. Simple
   - Provides a structured approach to the system implementation (involves breaking down the problem into parts)
2. Middle level language
   - It is used to develop system applications such as kernel , drivers etc
3. Provides a structural style
   - Involves implementing the program by breaking it into parts using functions that's makes ir easily to understand and modify the system
4. Rich Library
   - Provides a lot of inbuilt functions that make the development fast

Swift language

- A compile based & multi-paradigm programming language developed by Apple
- Used in developing Apple applications i.e. iOS , MacOS , tvOs and Linux
- Has better readability , high speed and dynamic libraries

R language

- Is an open source and interpreted programming language used for statistical computing (STATA)
- Used in financial domain in building statistical models
- Used among data miners for data analysis
- Used in analysing fraudulent transections
- R studio is the IDE used for R language

Kotlin language

- An android development language that is cross platform
- Its interoperability with java and support android studio which helps in writing less code with few bugs

Python

- An interpreted language involving writing the code in fewer lines as it consists of simple syntax
- Supports multiple programming models(paradigms) including object oriented programming , imperative style & functional programming

Features of Python

- User friendly
- Rich Library (integrated with C & C++ features)
- Provides cross browser support

USES OF Python language

- Used to create web applications
- Used in scientific & numerical computing
- Used in making desktop applications
- Used in creating social media sites i.e. Instagram and Pinterest

Scripting languages

- Languages that enhances program communication connectivity
- Scripting language does not require compiler and interpreter system
- Scripting languages fails under client side scripting language and server side scripting languages
- JavaScript , PHP are examples of scripting languages

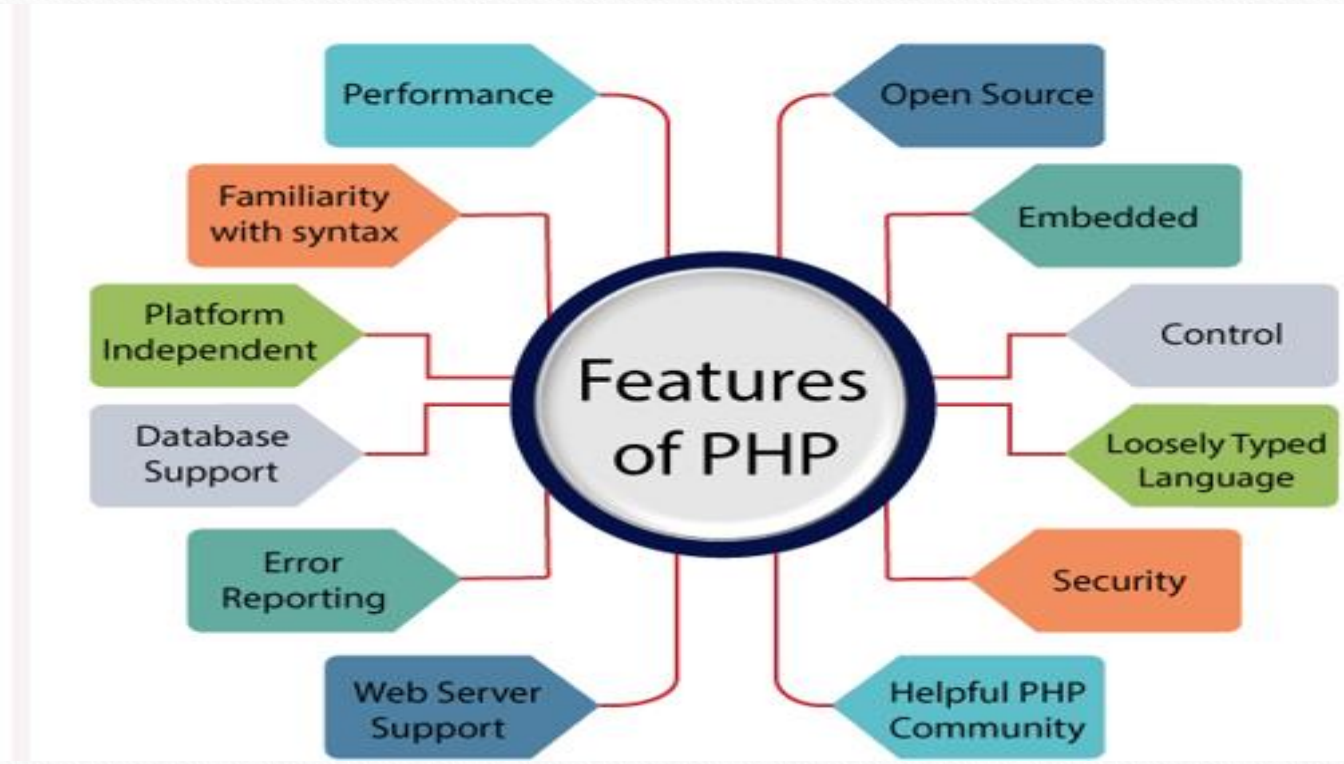
JavaScript

- A scripting language that enhances interactivity to clients side
- Web browser such as Mozira Firefox , Google chrome falls under client side
- It support object oriented and functional programming styles

PHP(Hypertext Pre-processor)

- It's server side scripting language used to design the dynamic web application with MYSQL database
- The language is used in developing web applications(an application that executes on the server & generates a dynamic page)
- Its an interpreted language meaning there is no need for compilation

Features of PHP



Features of PHP

1 High Performance

- PHP script is executed much faster as it uses its own memory hence the server loading time is automatically reduced

2 Open Source

- PHP source code is readily available on the web where one can modify it without costs or charges

3 Embedded (Simple)

- The code can be easily embedded with HTML tags and scripts

Features of PHP

4 Platform Independent

- PHP application developed in one OS can be easily executed in other OS
- PHP is available for WINDOWS, MAC OS, LINUX , UNIX operating system

5. Database Support

- Supports various databases such as MYSQL , SQLite ,ODBC etc.

USES (STRENGTH) OF PHP

- It handles the dynamic content, database & session tracking for the website
- Allows to create sessions and access session variables
- Helps to encrypt the data and apply validation
- Supports multiple protocols i.e. HTTP, SNMP,POP3 etc.

Compiled and Interpreted languages

- A compiler and an interpreter are programs that takes the source code and transforms it into machine code

Compiled Languages

- Programming languages comprising of semi-human readable features i.e. A=1+2 , B= 10*4
- Compiled language, program are directly converted into machine code or computer language (binary language)
- Compiled languages comprises of faster program execution speed
- Java , C# , C++ are examples of compiled languages
- Big programs such games , Operating systems are coded using compiled languages

STRENGTHS OF COMPILED LANGUAGES

- The source code is directly converted into machine code hence comprises of faster program execution speed
- It does not comprise of an intermediate program to translate the source code into machine code

WEAKNESS OF COMPILED LANGUAGES

- They are harder to program and require more efforts
- The language is not suitable for smaller programs

INTERPRETED LANGUAGES

- Programming languages in which an interpreter runs through a program line by line & execute each command
- In this language , the source code is not directly translated by the target machine, instead a different program (AKA) the interpreter ,reads and executes the code
- PHP, Perl ,Python , JavaScript , COBOL etc. are examples of interpreted languages

STRENGTHS OF INTERPRETED LANGUAGES

- Enhances program efficiency as each line of the code is debugged & executed
- comprises of an intermediate program(interpreter) converts the source code into machine code
- Suitable for smaller programs

WEAKNESS OF INTERPRETED LANGUAGES

- Comprises of slower program execution speed as an interpreter checks each line of the program code
- The language is suitable to smaller programs

Difference Between Source code and Machine code

Source code

- A list of human- readable instructions that a programmer writes
- The source code is written using a specific programming language i.e. Java , C# etc. and is converted into machine code.
- The source code comprises of human readable features i.e. Add , sum , average etc.
- The source code is converted into machine code language to be made executable and runs through a compiler

USING FLOW CHARTS AND PSEUDO-CODE TO DESCRIBE ALGORITHMS

1. PSEUDO-CODE

- Is a set of sequential human language instructions that describes the actions a program will take when coded in a particular programming language
- It uses human readable features to describe the program at high level without applying any programming language
- pseudo code helps programmers determine the step by step actions, a program must take to complete a required/desired task

PSEUDO-CODE that calculate average

1. Enter the first number
2. Enter the second number
3. Calculate sum
4. Divide sum by 2
5. Display the resuit

ADVANTAGES OF PSEUDO-CODE

- Allows to focus on the logic of an algorithm rather than the language specific features/syntax
- Since they are written in human language, eliminates the need of a specific programming language
- Enhances a formally checking , the code corrections on paper before implementing it using specific programming language

Algorithm

•Logical step by step instructions or procedures to solve a problem written in a particular programming language e.g. C#

•It uses specific programming language syntax to describe the program i.e. functions, data structures etc.

- Algorithms are converted into machine code by a program compiler

Algorithm to calculate average

1. Int num1 = 10;
2. Int num2 = 20;
3. Int sum;
4. Sum = num1 + num 2;
5. System.out.println(“ The average is” + sum);

FLOW CHART

•A diagram that represents program algorithm or processes

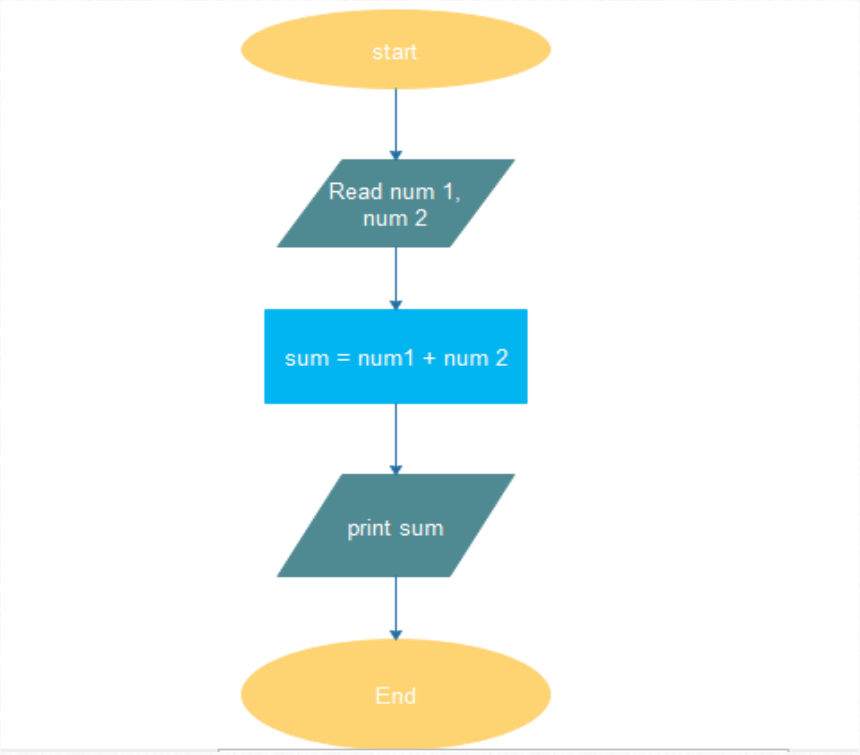
•Uses various symbols or notations in describing the program algorithm eg.

1. Start or stop = represents the first & last logic instruction of the program

2. Process = represents mathematical operations of the program

3. Flow line – arrows that shows the flow of instructions or process in a program

Flow chart to add two numbers entered by the user



THE USE OF TRACE TABLES TO SUPPORT CODE TRACING

Code tracing

- A process of assessing or examining thoroughness or accuracy of pseudo code

TRACE TABLES

- A technique or method used to test the logic of algorithms
- Allows to trace the value of variables as each line of code is executed
- Trace tables involves numbering algorithms or pseudo code in table rows and variables in table columns
- Trace tables helps in identifying or detecting errors in the algorithm easily

TRACE TABLE

Code line	num1	num2	sum	average	input	output
1	10				User enters first number	
2	10	20			User enters second number	
3	10	20	30		Adding first and second numbers	
4	10	20	30	30/2	Diving total by 2	
5	10	20	30	30/2		Average number 15 is displayed

Difference Between Source code and Machine code

Machine code

- Is the actual executable code derived from the source code
- Machine code comprises of binary bits i.e.1s and 0s (11100010)
- They are faster and efficient to execute as there are in the processors native language

Factors affecting the suitability of a language for particular tasks (e.g. client-side web scripting, server-side web scripting, mobile apps, scientific data processing, office applications, games, etc.)

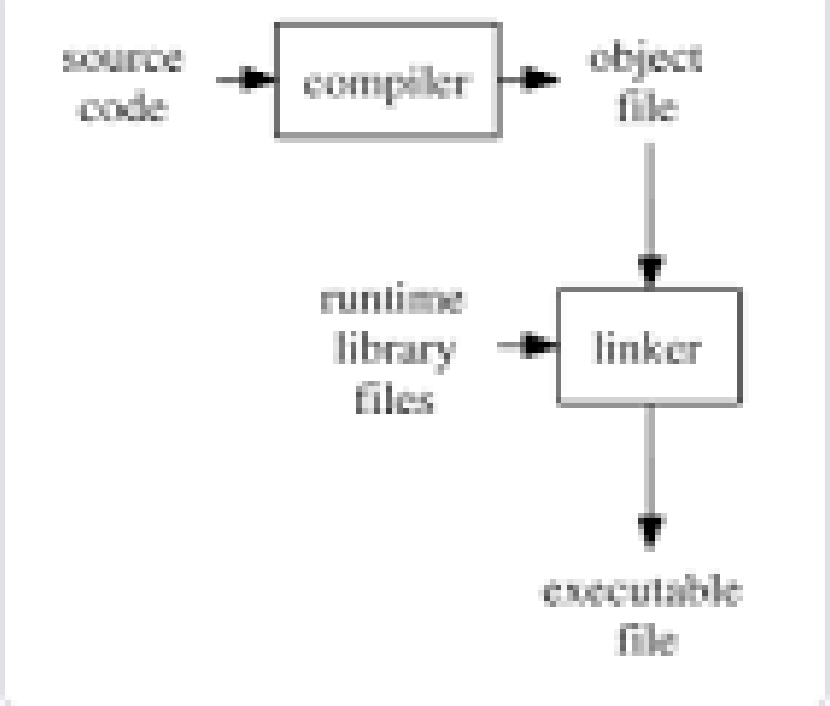
---

1. Execution speed – involves creating a program that carries out its operation faster
2. Robustness – the language should allow to authenticate users before using the program i.e. JavaScript (login authentication) PHP (sessions)
3. Security – the language should allow the program to withstand breaches i.e. brute force attacks by defining exceptional handlers and automatically shutting down the program
4. Interoperability – the language allows the program to run on any architecture
5. Modification – the language should allow the program to be updated or add new features
6. Deployment – the language should allow programs to be installed on any platform either mobile computers or cloud servers

Process of source code compilation

- A compiler takes the program code(source code) and converts the source code to machine language module (called an object file)
- Another program called a linker , combines the object file with other compiled object files
- Then an executable file is created

Process of source code compilation

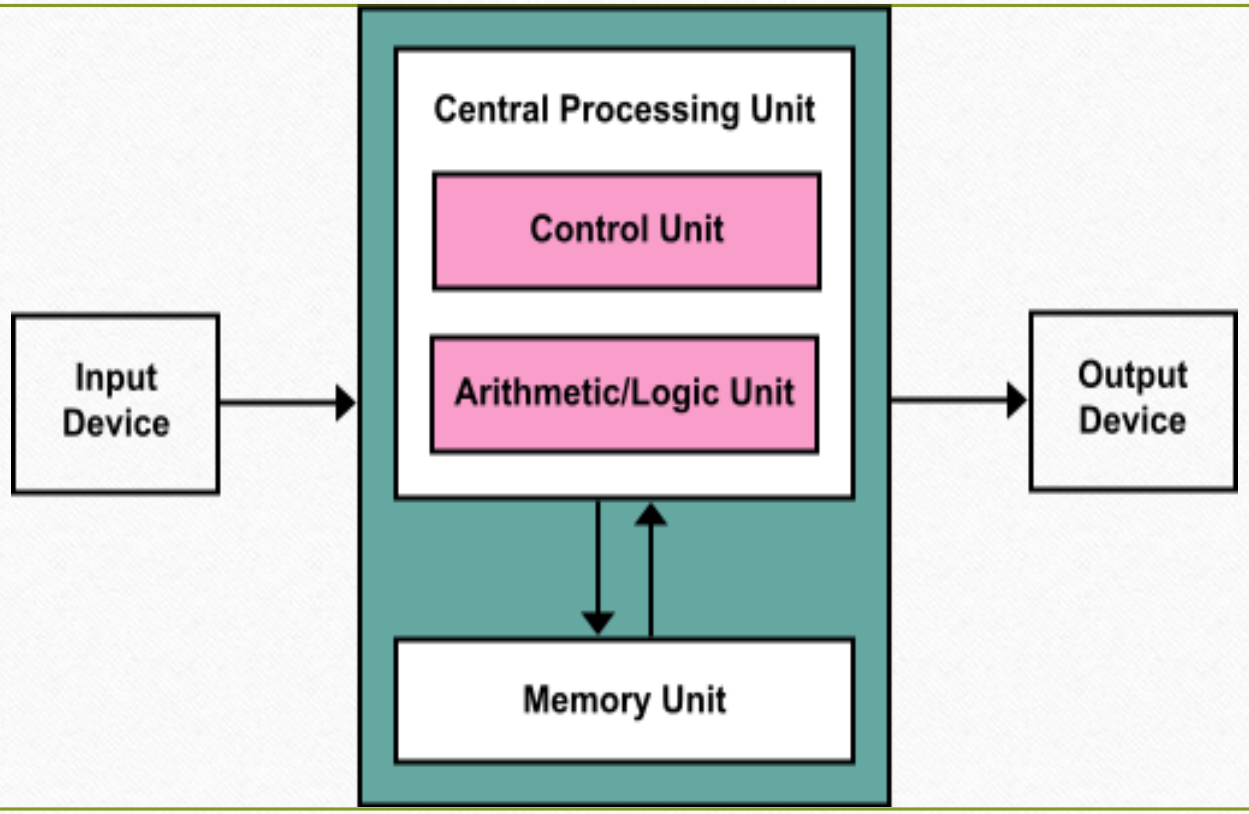


Principles of CPU architecture

Central Processing Unit (CPU)

- Is an electronic component within a computer system that carries out the instructions of a computer program
- It performs basic arithmetic , logical , control and Input/Output(I/O) operations specified by the instructions

Principles of CPU architecture



Principles of CPU architecture

1. **Control Unit:**

- Controls the operation of the ALU, memory, and input/output
  - Managing data flow between other components and the CPU
  - Decoding instructions
  - Storing the resulting data back into a memory unit

Principles of CPU architecture

2. **Arithmetic and Logical Unit (ALU):**

- Allows logical and arithmetic operations to be carried out such as addition and subtraction.
- The arithmetic logic unit executes all calculations within the CPU
- (Logical operators are: AND, OR, NOT, XOR)

Principles of CPU architecture

3. **Input/Outputs:**

- Information passed from the user/information received by the user.

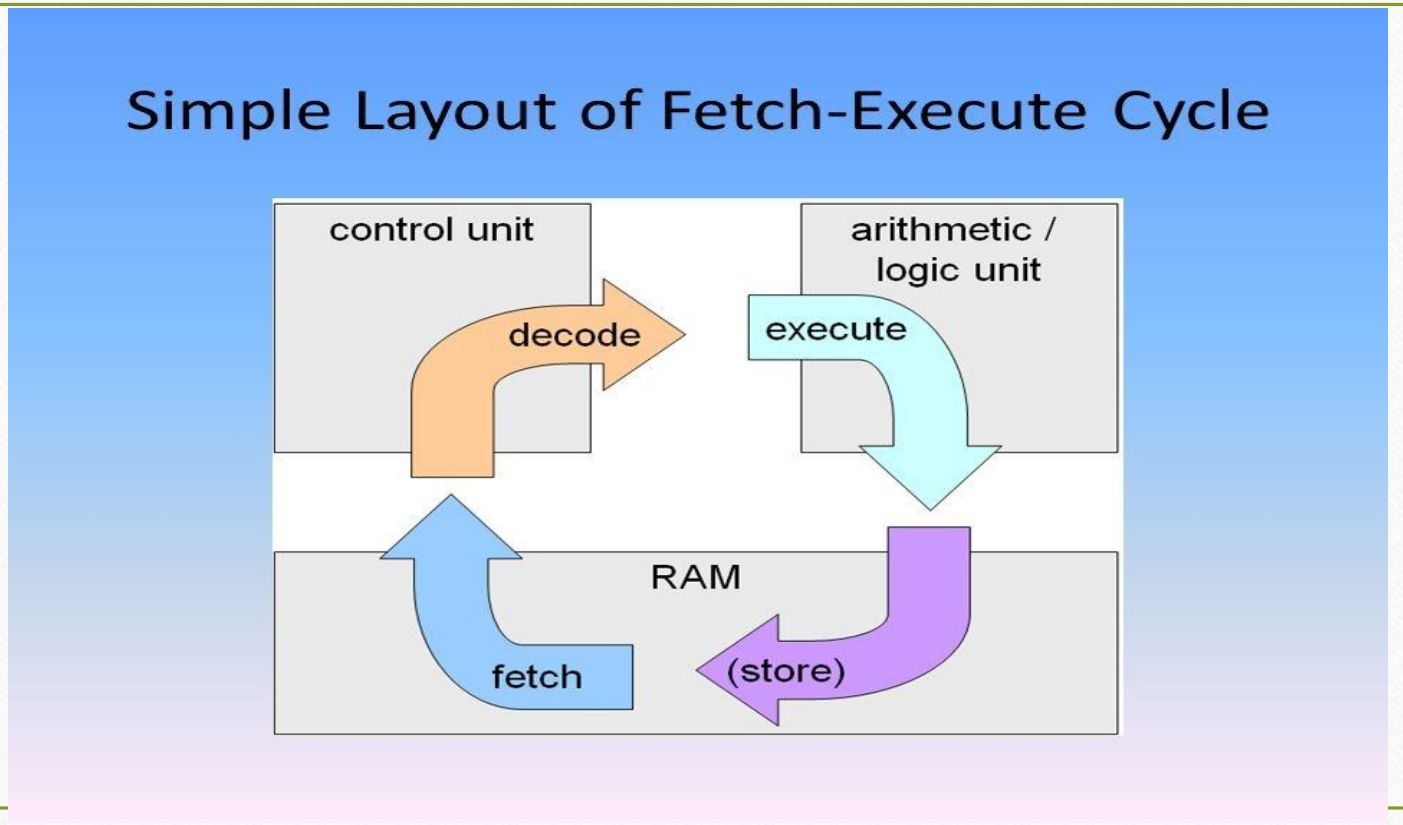
4. **Memory unit**

- Comprises of main memory and registers through which a computer boots
  - Register - a memory location within the actual processor that work at very fast speeds.
  - It stores instructions which await to be decoded or executed

The fetch-execute cycle

- It is the process by which a computer retrieves a program instruction from its memory and determines what actions the instruction requires
- The fetch –execute cycle is also called instruction cycle
- The cycle undertakes four major steps namely: Fetch, Decode , Execute and Store

The fetch-execute cycle



The fetch-execute cycle

1. Fetch

- The CPU retrieves or fetches some data and instructions from main memory
- Then stores these instructions in its own internal or temporary memory called registers

2. Decode

- Translates the retrieved instructions into a series of computer commands

The fetch-execute cycle

3. Execute

- Executes or triggers computer commands

4. Store

- Sends and writes the results back in memory

Apply common data types, data structures, and algorithms to problem solving

Common primitive data types (e.g. Boolean, float, double, integer, long, character, string, etc.)

Primitive data types

- These are predefined or inbuilt data types
- These data structures are manipulated directly from the machine instructions hence they are atomic
- Integer , double , float , char , pointers etc. are primitive data types

Primitive data types

1. Integer

- Stores whole numbers, positives or negatives(150 , -210) without decimals i.e.

```
Byte myNum=150;
System.out.println(myNum)
```

2. Float

- The data type that store numbers with a fractional part ,containing one decimal

```
Float myNum = 71.6;
System.out.println(myNum);
```

Primitive data types

3. Double

- The data type that stores numbers containing two decimals
- Variables such as money or currency need to be stored as double

```
Double myNum= 21.23;

System.out.println(myNum);
```

Primitive data types

5. Boolean

- A data type that stores or takes the values true or false
- Variables such as gender , marital status , pass rate need to be stored as boolean

```
Boolean isJavaFun = true;
Boolean isMangoTesty =false;
System.out.println(isJavaFun);
System.out.println(isMangoTesty);
```

Primitive data types

6. Character (char)

- The data type is used to store a single character or unicode
- The character must be surrounded by single quotes i.e. ‘A’ or ‘Z’
- The datatype takes alphabetical letters e.g. grade which is A or B , blood group etc.

Primitive data types

6. String

- The data type that stores a sequence of characters(text)
- String values are surrounded by double quotes i.e.”Mtisunge”
- Variables that represents name of person , city , country ,need to be stored as string

```
String homeVillage= ”Mghoma@”;
System.out.println(homeVillage);
```

7. Pointer

- Pointer is a data type that holds addresses of the other variable

Composite Data type

- Data types defined by the user and are not directly manipulated by machine instructions
- Composite data types are made-up of primitive data type values
- Arrays , stacks , queues , link lists , trees , graphs are examples of composite data types

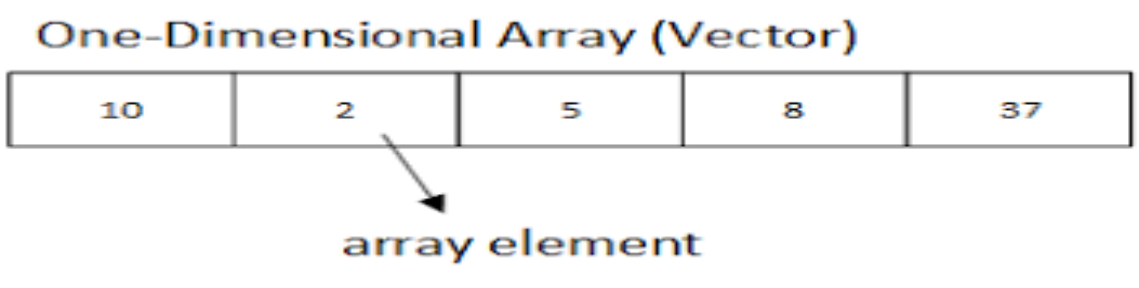
Composite Data type

1. Array Data type

- A big variable or container that stores large number of elements or data of same type
- Uses indexes to identify the position of an element in arrays
- The starting point of an array is 0 and values stored in an array are called elements
- Major forms of Arrays are one dimension array, two dimension arrays etc.

One Dimension Array

- An array consisting of length only no height



- The array above is of size [5] where element 2 is on index 1 and element 37 is on index 4

Syntax for Declaring and Entering values in One dimension array

1. Declaring one dimension array

- Involves defining **the array data type , array name & array size** e.g.

int scores [5]

2. Entering Values / Elements in one dimension array

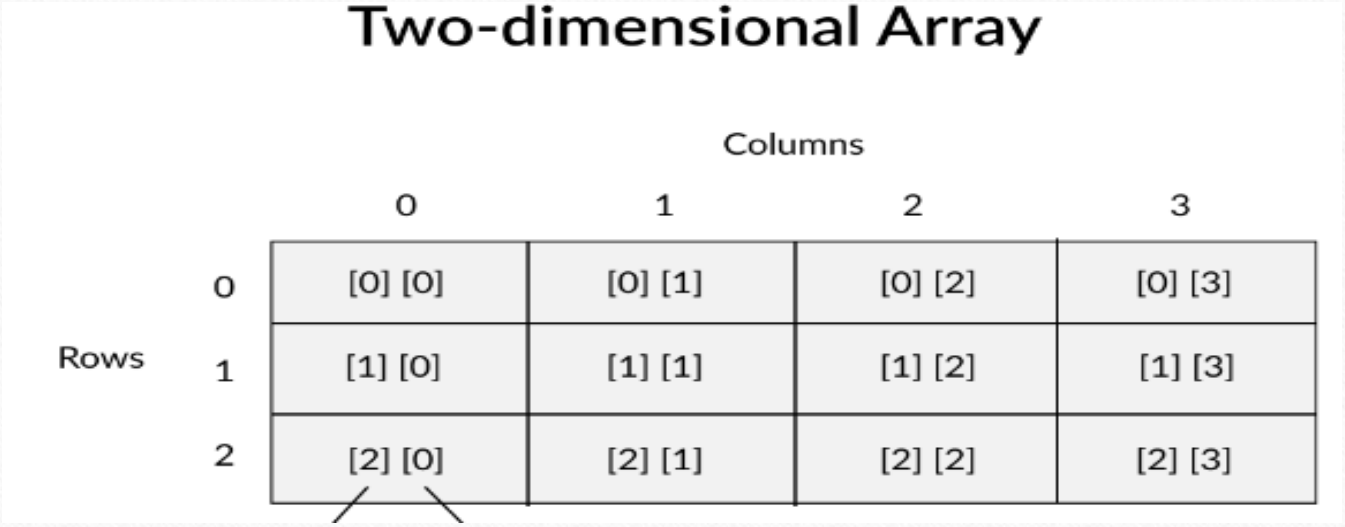
- Involves defining the **array name , the index where the value will be entered** and **the value to be stored**

Score [3] = 100 : value 100 will be stored on index 3 as shown below



Two Dimension Array

- An Array consisting of length and height
- Array a length is **row** and array height is **column**



Syntax for Declaring ,two Dimension Array

- Involves declaring the data type , array name , rows & columns e.g.

  INT S [4] [5]

Importance of Arrays

- Stores collection of elements or values that are all processed in the same way
- Used in creating a look up or description for things that are ordered or have non numeric names i.e. to pick a random day of the week

Limitation of Arrays

- The size of an array must be known in advance before using it in the program
- It is impossible to expand the size of an array during run time
- All elements in the array needs to be contiguously(connected without space or break) stored in the memory

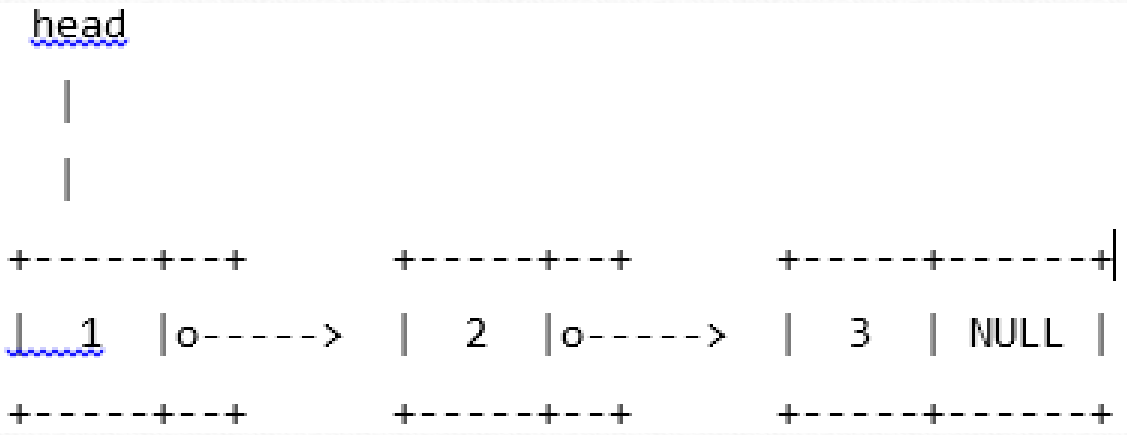
Composite Data type

2. Linked List

- a data structure made of a chain of nodes connecting to each other through pointer
- Each nodes contains actual data being stored and a pointer(link) to other nodes
- A pointer contains address of the next nodes in the memory



Types of Linked list



1. **Singly Linked List**

- Data type that contain nodes which have a data field as well a next field which points to the next node in sequence
- Singly link list travel only in one direction & each node contains only one next pointer
- Operations that are performed on singular linked list are insertion , deletion etc.

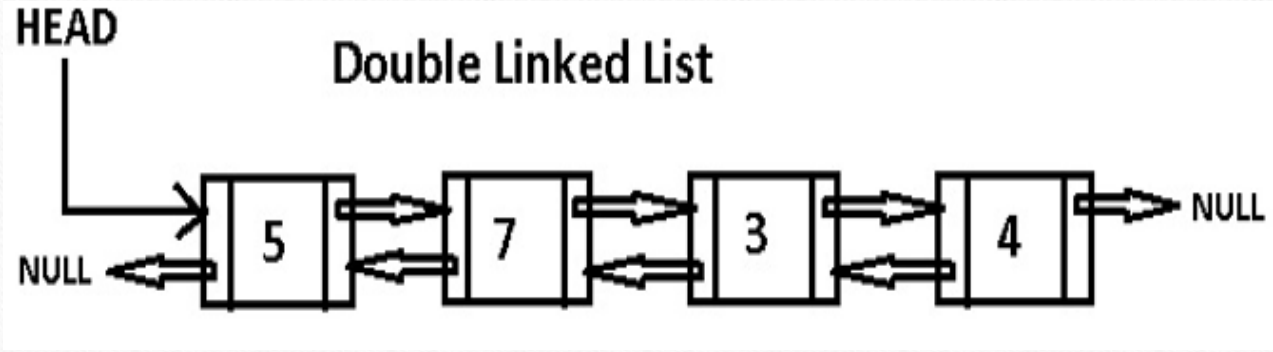
Types of Linked list

2. **Doubly Linked List**

- a data structure in which a node contains a pointer to the previous as well as the next node in the sequence

Anode in a double link list consist of

- Node data
- Pointer to the next node
- pointer to the previous node



Strength of Doubly Linked List

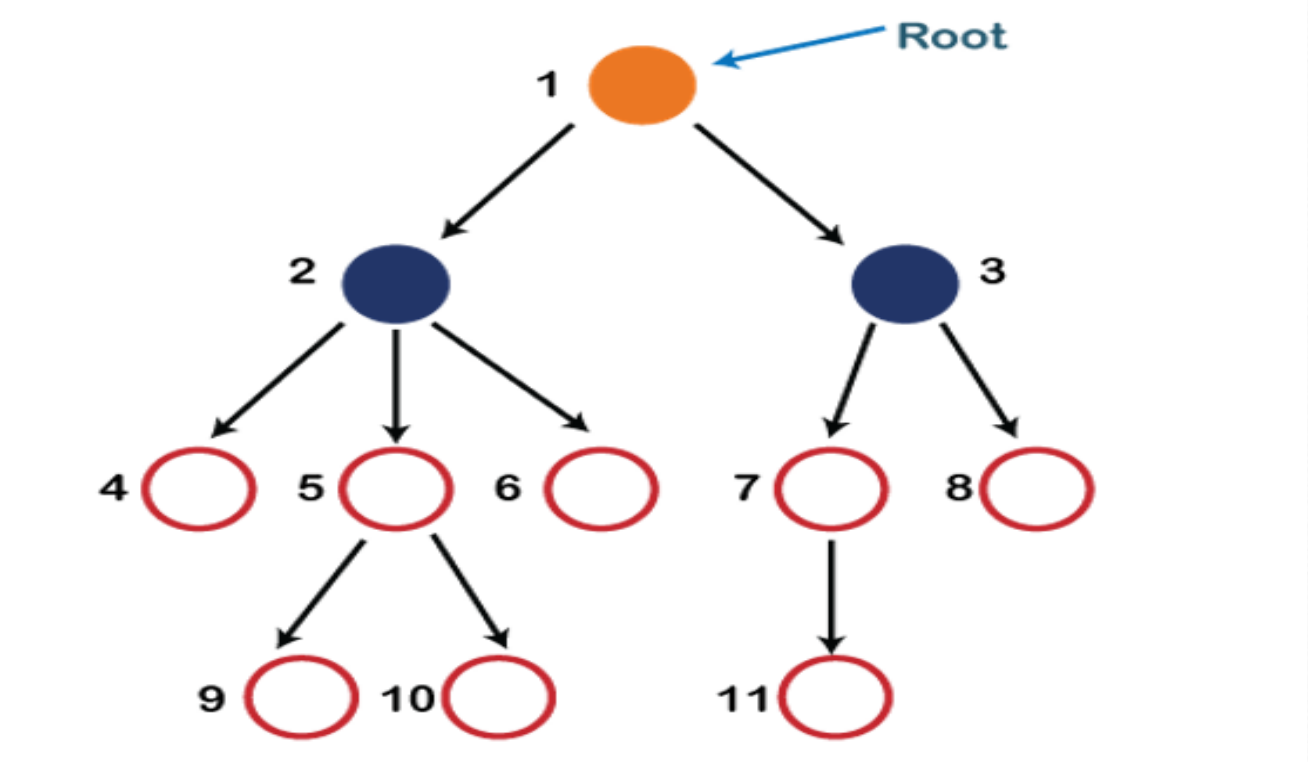
- Doubly linked has 2 pointers , which means it easier to move in forward and backward direction.
- This can make things simpler for scenarios where updating or deleting is frequent
- Use doubly linked list if a data structure is needed to work as a Queue and a Stack at the same time

Composite Data type

3. Tree Data Structure

- A tree data structure is defined as a collection of objects or entities known as nodes that are linked together to represent or simulate hierarchy. OR (A tree is a nonlinear hierarchical data structure that consists of nodes connected by edges)
- The data structures represent hierarchical data ,arranged in multiple levels.
- In the Tree data structure, the topmost node is known as a root node and is connected by the edge
- Each node contains some data, and data can be of any type. (**Node** : A node is an entity that contains a key or value and pointers to its child nodes)

. Tree Data Structure



Components of Tree Data Structure

1. **Root**

- The root node is the topmost node in the tree hierarchy. hence, the root node is the one that doesn't have any parent. E.g. node numbered 1 is **the root node of the tree**

2. Parent

- If the node contains any sub-node, then that node is said to be the parent of that sub-node

Components of Tree Data Structure

3. **Child node**

- If the node is a descendant of any node, then the node is known as a child node

4. **Sibling**

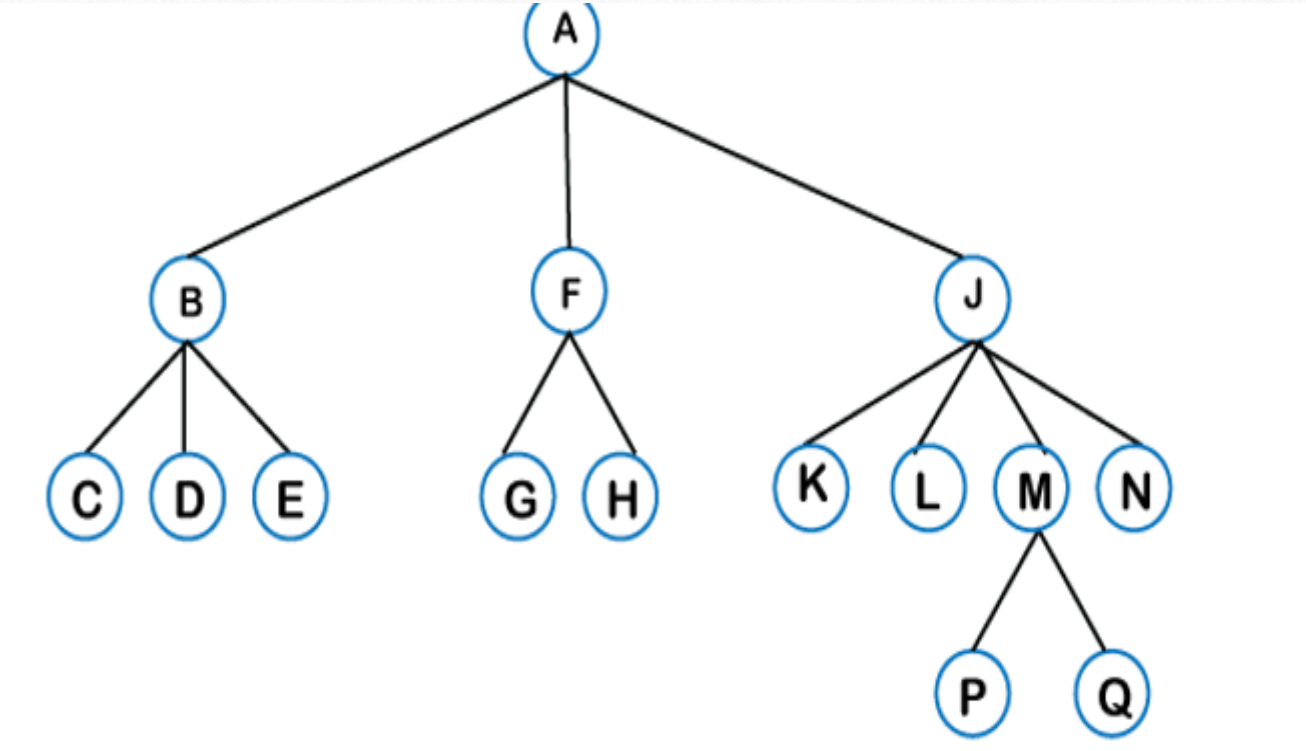
- The nodes that have the same parent are known as siblings

Types of Tree data structure

1. **General tree**

- is the data structure in which a node can have either 0 or maximum number of nodes.
- There is no restriction imposed on the degree of the node (the number of nodes that a node can contain).
- The topmost node in a general tree is known as a root node. The children of the parent node are known as ***subtrees***.

General tree

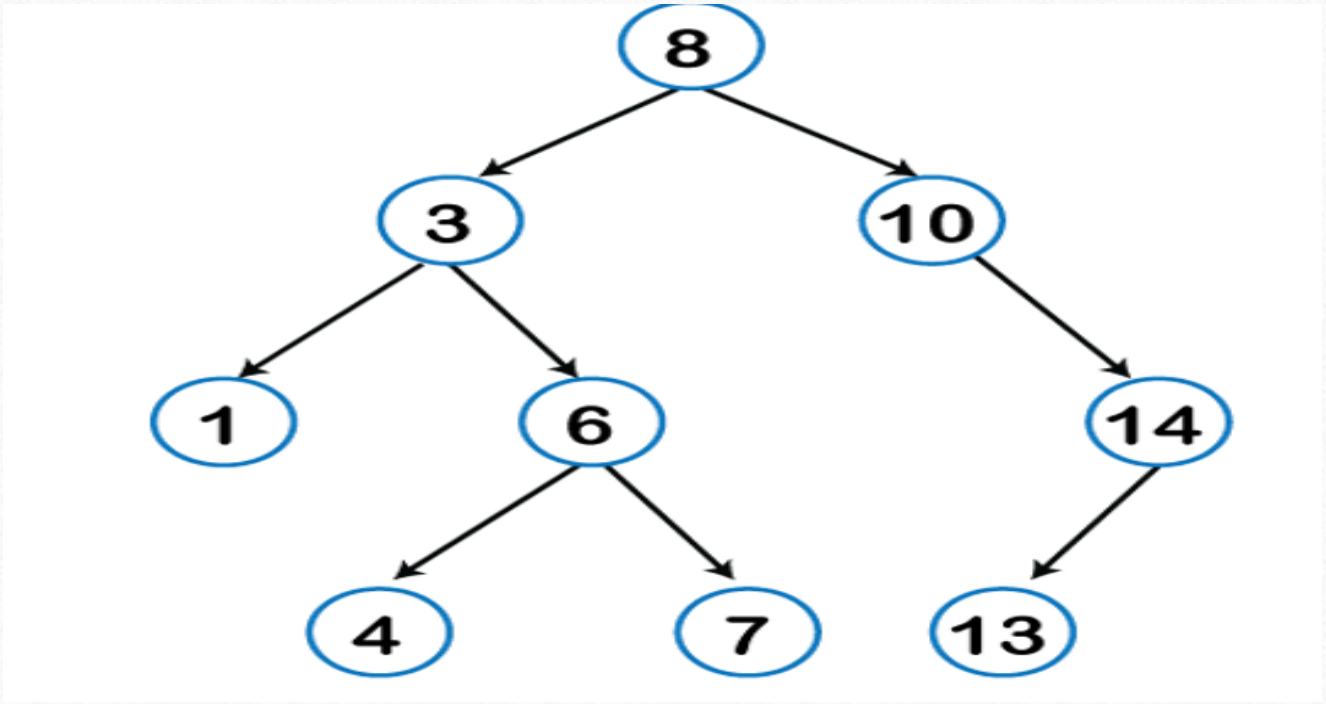


Types of Tree data structure

2. **Binary tree**

- Here, binary name itself suggests two numbers, i.e., 0 and 1.
- In a binary tree, each node in a tree can have utmost two child nodes
- . Here, utmost means whether the node has 0 nodes, 1 node or 2 nodes

Binary tree



Advantages of Tree

- It offers an efficient search and insertion procedure.
- The tree is used for storing hierarchy data
- Reduces time complexity in searching / inserting data

NOTE:

- Other data structures such as arrays, linked list, stack, and queue are linear data structures that store data sequentially.
- In order to perform any operation in a linear data structure, the time complexity increases with the increase in the data size which bad for modern computing

Common programming constructs (e.g. selection, iteration, recursion, function call, etc.)

Sequencing

- This means that the computer will run your code in order, one line at a time from the top to the bottom of your program.
- It will start at line 1, then execute line 2 then line 3 and so on till it reaches the last line of your program.

Common programming constructs (e.g. selection, iteration, recursion, function call, etc.)

Selection

- A constructs that allows some lines of code to be run only if a condition is met
- If statements defines selection construct
- An IF THEN statement is used to choose between additional or exclusive course of actions

SYNTAX FOR THE IF STRUCTURE

If <condition> then

Conditional code

End if

SYNTAX FOR THE IF STRUCTURE

```
If (marks >= 10){
  system.out.println('F');
Else{'D};
```

Common programming constructs (e.g. selection, iteration, recursion, function call, etc.)

3. Iteration:

- Is the repetition of a block of instructions (code) within a computer program for a number of instances or until status is encountered
- allows the computer to execute the same lines of code several times , using a loop.
- Iteration is implemented in programming using **For loops** and **WHILE** loops statements.

Iteration

For example, a very simple algorithm for eating breakfast cereal might consist of these steps:

1. put cereal in bowl
2. add milk to cereal
3. spoon cereal and milk into mouth
4. repeat step 3 until all cereal and milk is eaten
5. rinse bowl and spoon

- The algorithm will repeat steps 3 and 4 until all the cereal and milk has been eaten.

Types of Iteration

1. For loops (**Count-controlled loops)**

- loops that iterates steps a specific number of times
- It is used when the number of iterations to take place is already known.
- It uses a counter to keep track of how many times the set of commands has been repeated.
- Count-controlled loops are executed using FOR statements

For loops (**Count-controlled loops)**

- Syntax for a For loop

```
for (initializer; condition; iterator)
{
    //code block
}
```

1. **Initializer:**

- The initializer gives the starting value to the local variable
- local variables cannot be accessed outside loop

Syntax for a For loop

2. **Condition**

- The condition is a Boolean expression that will return either true or false.
- If an expression evaluates to true, then it will execute the loop again; otherwise, the loop is exited

3. **Iterator**:

- The iterator defines the incremental or decremental of the loop variable

Example of For loop

```
for(int i = 0; i < 10; i++)
{
    Console.WriteLine("Value of i: {0}", i);
}
```

2. While loops(**Condition-controlled loops)**

- Loops that iterates steps continuously until a condition is met.
- It is used when the number of iterations to take place is unknown.
- The algorithm tests the condition to see if it is true.
- If true, the algorithm executes a command.
- If false, the algorithm goes back to step 1 and will continue to go back until the condition becomes true
- Condition-controlled loops are executed using WHILE statements.

Syntax for While loops(**Condition-controlled loops)**

- The while loop executes a block of code when the specified condition return false

```
While(condition)
{
    //code block
}
```

- The while loop starts with the key word while & it must include a Boolean conditional expression that returns either true or false
- It executes the code block until the specified conditional expression returns false

Syntax for While loops(**Condition-controlled loops)**

- The initialization in while loop , is done before the loop starts
- The incremental or decremental is done inside the loop i.e.

```
int i = 0; // initialization

while (i < 10) // condition
{
    Console.WriteLine("i = {0}", i);

    i++; // increment
}
```

if Statement

- The if statement contains a boolean condition followed by a single or multi-line code block to be executed.
- It is executed if condition is true.

```
if(condition)
{
    // code block to be executed when if
condition evaluates to true
}
```

Example of if statement

```
int i = 10, j = 20;

if (i < j)
{
    Console.WriteLine("i is less than j");
}

if (i > j)
{
    Console.WriteLine("i is greater than j");
}
```

Example 2 : IF Statement

```
1. using System;
2. public class IfExample
3.     {
4.         public static void Main(string[] args)
5.         {
6.             int num = 10;
7.             if (num % 2 == 0)
8.             {
9.                 Console.WriteLine("It is even number");
10.                 }
11.
12.                 }
13.             }
```

else Statement

- The statement does not contain any condition
- Contains a statement that is executed when an IF and IF else conditions are false

```
int i = 20, j = 20;

if (i > j)
{
    Console.WriteLine("i is greater than j");
}
else if (i < j)
{
    Console.WriteLine("i is less than j");
}
else
{
    Console.WriteLine("i is equal to j");
}
```

Example 2: IF-Else

```
1.  using System;
2.  public class IfExample
3.      {
4.          public static void Main(string[] args)
5.          {
6.              int num = 11;
7.              if (num % 2 == 0)
8.              {
9.                  Console.WriteLine("It is even number");
10.             }
11.             else
12.             {
13.                 Console.WriteLine("It is odd number");
14.             }
15.
16.         }
17.     }
```

Output:

```
It is odd number
```

If-else Example: with input from user

- In this example, we are getting input from the user using **Console.ReadLine()** method.
- It returns string. For numeric value, you need to convert it into int using **Convert.ToInt32()** method.

If-else Example: with input from user

```
1.  using System;
2.  public class IfExample
3.      {
4.          public static void Main(string[] args)
5.          {
6.              Console.WriteLine("Enter a number:");
7.              int num = Convert.ToInt32(Console.ReadLine());
8.
9.              if (num % 2 == 0)
10.             {
11.                 Console.WriteLine("It is even number");
12.             }
13.             else
14.             {
15.                 Console.WriteLine("It is odd number");
16.             }
17.
18.         }
19.     }
```

Output:

```
Enter a number:11
It is odd number
```

Output:

```
Enter a number:12
It is even number
```

IF-else-if ladder Statement

- if-else-if ladder statement executes one condition from multiple statements

Syntax

```
1. if(condition1){
2. //code to be executed if condition1 is true
3. }else if(condition2){
4. //code to be executed if condition2 is true
5. }
6. else if(condition3){
7. //code to be executed if condition3 is true
8. }
9. ...
10.     else{
11.     //code to be executed if all the conditions are false
12.     }
```

IF-else-if ladder Statement

```
1.  using System;
2.  public class IfExample
3.      {
4.          public static void Main(string[] args)
5.          {
6.              Console.WriteLine("Enter a number to check grade:");
7.              int num = Convert.ToInt32(Console.ReadLine());
8.
9.              if (num <0 || num >100)
10.                 {
11.                     Console.WriteLine("wrong number");
12.                 }
13.                 else if(num >= 0 && num < 50){
14.                     Console.WriteLine("Fail");
15.                 }
16.                 else if (num >= 50 && num < 60)
17.                 {
18.                     Console.WriteLine("D Grade");
19.         |           }
```

IF-else-if ladder Statement

```
20.         else if (num >= 60 && num < 70)
21.         {
22.             Console.WriteLine("C Grade");
23.         }
24.         else if (num >= 70 && num < 80)
25.         {
26.             Console.WriteLine("B Grade");
27.         }
28.         else if (num >= 80 && num < 90)
29.         {
```

IF-else-if ladder Statement

```
30.                 Console.WriteLine("A Grade");
31.             }
32.             else if (num >= 90 && num <= 100)
33.             {
34.                 Console.WriteLine("A+ Grade");
35.             }
36.         }
37.     }
```

Output:

```
Enter a number to check grade:66
C Grade
```

Output:

```
Enter a number to check grade:-2
wrong number
```

Switch Statement

- Statement used to test a variable against three or more conditions
- Switch statement is used an auto native to IF Else statement
- executes one statement from multiple conditions

Switch Statement Syntax

```
1.  switch(expression){
2.  case value1:
3.   //code to be executed;
4.   break;
5.  case value2:
6.   //code to be executed;
7.   break;
8.  ......
9.
10. default:
11.  //code to be executed if all cases are not matched;
12.  break;
13. }
```

Switch Statement

- The switch statement starts with the switch keyword that contains an expression or variable in brackets
- The result of the expression will be tested against conditions specified as cases
- A case must be specified with the unique constant value & ends with the colon :
- The case will be executed if a constant value & the value of expression/varable are equal
- The defauit key word will be executed if no cases executed while break/ return keyword are used to exit the program from switch case

Example switch statement 1

```
1.  using System;
2.    public class SwitchExample
3.     {
4.       public static void Main(string[] args)
5.       {
6.           Console.WriteLine("Enter a number:");
7.           int num = Convert.ToInt32(Console.ReadLine());
8.
9.           switch (num)
10.                 {
11.                    case 10: Console.WriteLine("It is 10"); break;
12.                    case 20: Console.WriteLine("It is 20"); break;
13.                    case 30: Console.WriteLine("It is 30"); break;
14.                    default: Console.WriteLine("Not 10, 20 or 30"); break;
15.                 }
16.              }
17.           }
```

Output:

```
Enter a number:
10
It is 10
```

Output:

```
Enter a number:
55
Not 10, 20 or 30
```

Example of **Switch Statement**

```
int x = 10;

switch (x)
{
     case 5:
          Console.WriteLine("Value of x is 5");
          break;
     case 10:
          Console.WriteLine("Value of x is 10");
          break;
     case 15:
          Console.WriteLine("Value of x is 15");
          break;
     default:
          Console.WriteLine("Unknown value");
          break;
}
```

Try it

Output:

```
Value of x is 10
```

3. Recursion

- **a process in which a function calls itself**
- This allows the function to be repeated several times, since it calls itself during its execution
- Recursion requires the least amount of code to perform the necessary functions
- Recursive functions allow programmers to write efficient programs using a minimal amount of code

Example of Recursive function

```
function Count (integer N)
    if (N <= 0) return "Must be a Positive Integer";
    if (N > 9) return "Counting Completed";
    else return Count (N+1);
end function
```

Example of Recursive function

- recursive method needs to be terminated, therefore, we need to write a condition in which we check is the termination condition satisfied.
- If we don't do that, a recursive method will end up calling itself endlessly
- Example : Create an application which calculates the sum of all the numbers from n to m recursively:

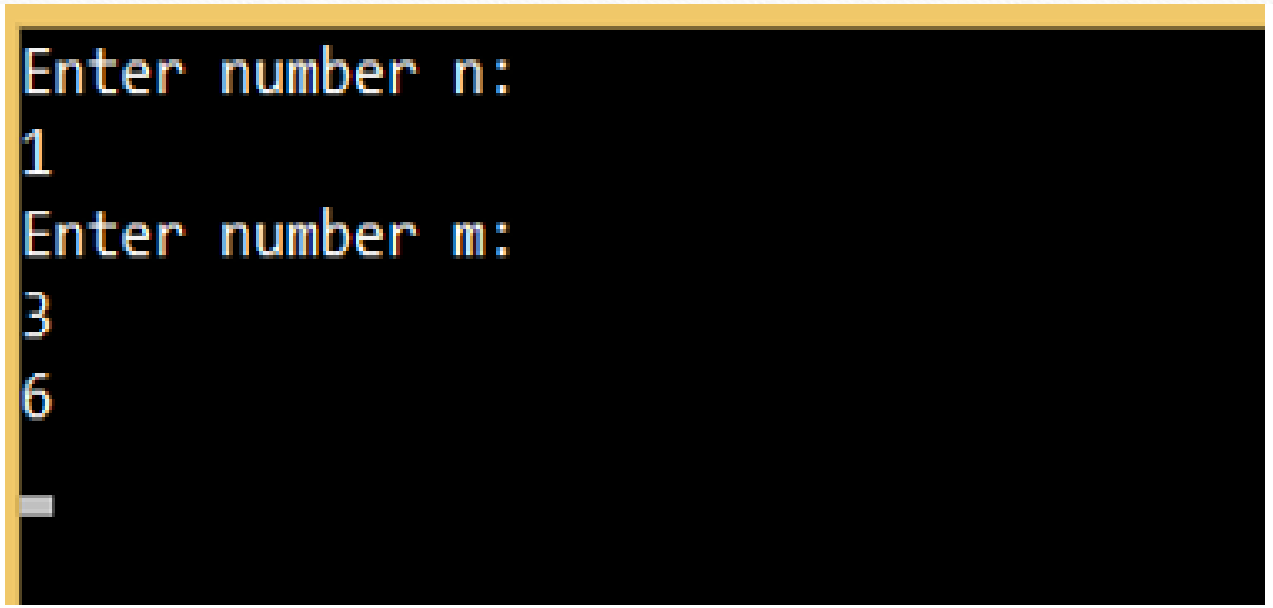
Example of Recursive function

```
class Program
{
public static int CalculateSumRecursively(int n, int m)
{
int sum = n;
if(n < m)
{
n++;
return sum += CalculateSumRecursively(n, m);
```

Example of Recursive function

```
}
return sum;
}
static void Main(string[] args)
{
Console.WriteLine("Enter number n: ");
int n = Convert.ToInt32(Console.ReadLine());
Console.WriteLine("Enter number m: ");
int m = Convert.ToInt32(Console.ReadLine());
int sum = CalculateSumRecursively(n, m);
Console.WriteLine(sum);
Console.ReadKey();
}
```

OUTPUT



Code Explanation

The method `CalculateSumRecursively` is our recursive method that calculates the sum of the numbers from `n` to `m`. The first thing we do is to set our `sum` to the value of `n`. Then, we check if the value of `n` is less than the value of `m`. If it is, we increase the value of `n` by 1 and add to our `sum` a result

Functions, arguments, and return values

1. Function

- is a code block that contains a series of statements that performs certain tasks OR
- is a small or mini section of the code used to perform a specific task e.g. adding two numbers
- Functions could be of parameterised and non-parameterised , parameterised functions consists of function signatures(arguments)
- Function facilitates the easier and faster implementation of the program

Function

```
Function addTwoNumbers( num 1 as whole number , num2 as whole number)
    Data ans as whole number
    Ans = num 1 + num 2
    Output ‘ the answer is’ + ans
    Return ans
End function
```

Syntax in writing function

1. Start with the key word Function
2. Indent instructions or statements
3. End with key word End Function

Arguments

- Values contained in the body of the function or method
- Arguments are enclosed within the method open & closing brackets (num1 , num 2)
- Arguments are also called method signatures or parameters
- Specifies the type of the function to be involved or called in the program
- Function addTwoNumbers has arguments such as num1 , num 2

Arguments

```
int Square(int i)
{
    // Store input argument in a local variable.
    int input = i;
    return input * input;
}
```

- Int i is an argument for the function square

RETURN VALUES

- Values the function or method returns to the main program after calculations
- Return values are mostly known values namely as integers , characters , Boolean , strings , long depending on the type of the program

```
int Square(int i)
{
    // Store input argument in a local variable.
    int input = i;
    return input * input;
}
```

Principles and history of object-oriented programming

- Object Oriented programming (OOP) is a programming paradigm that relies on the concept of **classes** and **objects**
- It is used to structure a software program into simple & reusable pieces of code
- Object-oriented languages help to manage complexity in large programs

history of object-oriented programming

- SIMULA I (1962-65) and Simula 67 (1967) are the two first object-oriented languages.
- Simula 67 introduced most of the key concepts of object-oriented programming: both objects and classes, subclasses (usually referred to as inheritance)
- OOP was invented by Ole-Johan Dahl and Kristen Nygaard on the design of the SIMULA language,which started at the Norwegian Computing Centre in the Spring of 1961

Principles object-oriented programming

1. Object
2. Class
3. Inheritance
4. Polymorphism
5. Abstraction
6. Encapsulation

Principles object-oriented programming

1. An Object

- an entity that has characteristics(properties) and behaviour e.g. student
- Behaviour defines an object by the way it interacts with the system
- Objects can correspond to real-world objects or an abstract entity

Principles object-oriented programming

2. Class

- A template that models system objects
- Classes are the fundamental building blocks of a program
- Contains descriptions of all the behaviors and properties of the system objects

Class Declaration

- Involves

1. Specifying a name for the class (student)
2. Defining the behaviors for the class(methods)
3. Class relationships i.e.

```
Public class student {
   Int age;
   String address;
}
```

Class Declaration

- **Class** name is a unique name that defines the container that model program object e.g. student class
- Attributes – are properties or characteristics of program objects i.e. age , id etc.
- **Methods** are functions that are defined inside a class that describe the behaviors of an object.
- Methods specifies operations carried out by objects

Principles object-oriented programming

3. Inheritance

- a framework that involves child classes acquiring all the properties and behaviors of the supper class
- Inheritance allows classes to reuse code from other classes
- Common reused codes in classes are : functions , variables etc.

Inheritance in Java

- Java uses keyword extend to implement inheritance among classes

```
Public class A {
        Public void write () {
            String name;
          }
   }
Public class B extends A {
   Public void write () {
     String name;
   }
}
```

Principles object-oriented programming

4. Encapsulation

- a mechanism that restricts direct access to some of the objects or functions of the class (data hiding)
- involves binding code and data together into a single unit hence objects of the class are only accessed by the methods of that class
- Encapsulation hides the internal software code implementation inside a class
- in encapsulation the variables of the class are declared as private and public & setter and getter methods are used to modify and view variable values i.e.

Encapsulation Test

```
Public class EncapTest{
  Private string name;
  Private string id;
  Private int age;

  Public int get age(){
       Return age;
  }

  Public string setName(){
      Return name;
  }

  Public string getId(){
     Return id;
  }

  Public void setAge(int newAge){
       Age= newAge;
  }
  Public void setName(string newName){
      Name = newName;
  }
```

ACCESSING ENCAPTEST CLASS

```
Public static void main (string[]args){
    EncapTest encap = new EncapTest();
    Encap.setName('mtisunge');
 Encap.setAge(12);
 Encap.setId('m0012');
System .out.println('Name'+ encap.getName())
}
```

BENEFITS OF ENCAPSULATION

- Fields of a class can be made read only or write only
- A class can have total control over what is stored in its fields

Hides complexity: No one can see what's behind the algorithm

Principles object-oriented programming

5. Polymorphism

- the ability of an object to take many forms that gives away to access it easily
- Objects are designed to share behaviors and they can take on more than one form
- Method overriding and overloading is an example of polymorphism

Polymorphism

1. Method overloading

- is a process where there are more methods in a class of the same name, with different parameters or method signature
- Associated with two methods with the same name but different parameter

Method overloading

```
Public class student {
    Public void write () {
        String name;
      }
     Public void write (int i) {
    }
  }
```

Method overriding

- a mechanism where two methods with the same name , same parameters but in two different classes
- Method overriding involves one class surpassing a method of the other class inherited i.e.

Method overriding

```
Public class A {
        Public void write () {
            String name;
          }
   }
Public class B extends A {
   Public void write () {
      String name;
   }
}
```

Advantages of OOP

- **Code Maintenance** - Provides a clear flow of the system algorithm hence makes easier to develop and maintain the code mainly when the project size increases
- **improves Security** -Enhances system robustness through data hiding concept
- **Faster development** - Saves time when coding the system since most of the objects , functions , classes can be inherited

DISADVANTAGES OF OBJECT ORIENTED PROGRAMING

- **Slower programs**: Object-oriented programs are typically slower than procedure-based programs, as they typically require more instructions to be executed
- **Difficult to implement :** Object-oriented programs typically involve more lines of code than procedural programs (Larger program size)

Functional programming

- A programming paradigm that involves writing codes into smaller blocks called functions
- Functional programming is associated with the ,effect of achieving the same tentative results when given the same inputs
- The methodology involves understanding what the result is rather than how we get the result
- Concentrates only on the program inputs and outputs

Characteristics of Functional Programming

- Functional programming method focuses on results, not the process
- Emphasis is on what is to be computed
- Data is immutable
- Functional programming Decompose the problem into ‘functions

Benefits of functional programming

- Easier to test and execute Unit testing and debug FP Code.
- Enhances Parallel processing (execution of codes in different blocks)
- Supports Nested Functions

Limitations of Functional Programming

- Functional programming paradigm is not easy, so it is difficult to understand the syntax
- Re-use is very complicated and needs constantly refactoring
- Objects may not represent the problem correctly

Imperative programming language

- A paradigm consisting of commands or clearly defined sequence of instructions for the computer to perform
- Focuses on describing how a program operates hence the code is easy to understand
- Describes which steps are to be performed in order to obtain the desired solution at the end
- The source code for imperative language is a series of commands which specify what the computer has to do and when in order to achieve a desired result
- Java , ALGOL, Pascal , C , C++, C# , Ruby are some of imperative programming languages

Advantages of Imperative programming

- Easy to read and learn – consists of step by step instructions which are easy to follow or debug
- Conceptual model is very easy for beginners to understand

Disadvantages Imperative Programming

- Difficult to debug in context where the volume of the code starts to glow i.e. in procedural approach it difficult to trace errors in complex programs
- There is higher risk of errors when modifying the program

Object-orientated design considerations (e.g. visibility modifiers, coupling versus cohesion, etc.)

1) VISIBILITY MODIFIER

- Mechanism that sets the accessibility(visibility)of classes , objects ,constructors & functions in a language•
- Prevents unauthorized access of codes (methods) in classes
- Visibility modifiers are categorized into, private, public, protected etc.

VISIBILITY MODIFIER

1. Private access modifier

•A modifier that restricts or hides data from the outside class

Private access modifiers uses a negative (-) notation

•Methods , variables , that are declared private ,can only be accessed within the class declared

•Variables that are declared private can be accessed outside the class ,if public getter methods are present in the class

Private access modifier

```
Public class logger{
  Private string format;
  Public string getFormat(){
     Return this.format;
     }
  Public void setFormat(string format){
     This.format=format;
     }
  }
```

Private access modifier

NOTE:

The format variable of logger class is private , so there is no way for other classes to retrieve or set its value directly , to make the variable available to the outside world , define two public methods : getformat() which returns the value of format and setformat(string) which sets its value.

VISIBILITY MODIFIER

II. Public access modifier

•A modifier that provides direct access of methods , variables ,constructors to any class of the program

Public access modifier uses a positive (+) notation or symbol

•The main()/ method of an application has to be public otherwise it could be not called by the language interpreter to run the class

Public access modifier

```
Public class student{
    public string address;
    public int age;
    console.out.println("please enter your address");
```

VISIBILITY MODIFIER

III Protect access modifier

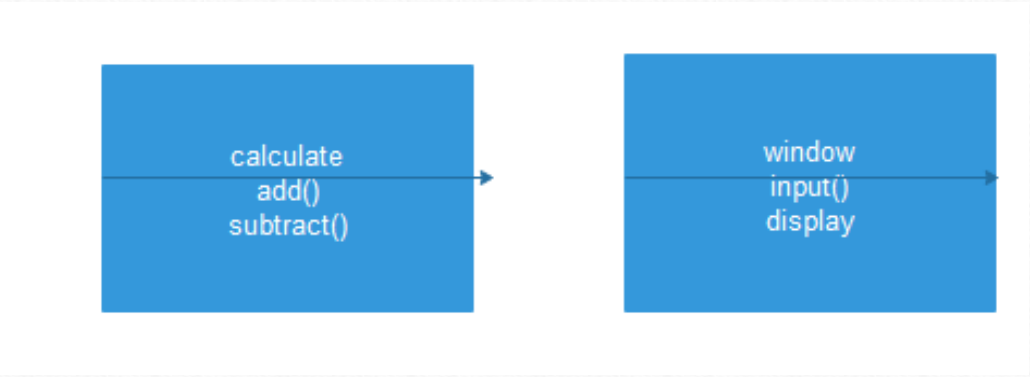
- Involves accessing variables , methods and constructors declared within the package
- Variables , methods and constructors declared protected can be called by any other class within the same package or sub class of other package
- Protected access modifier cannot be applied to class and interfaces

2. COHENSION

- The extent to which a component has a single purpose or function
- Cohesion falls under two major forms namely: High cohesion & low cohesion

High COHENSION

- cohesion in which the code is distributed into classes(there are more classes)
- High cohesion is good because they are easily to reuse
- High cohesion brings about more robustness and reliability



Low COHENSION

- cohesion in which the code is distributed or defined in a single / one class
- Low cohesion leads to less reusability of the code and less testability
- Low cohesion makes difficult to debug the program (algorithm)



3. COUPLING

- The extent to which two components depend on each other for successful execution
- High coupling – a module(class) has many relationships with other modules
- Low coupling – a module has fewer relationships with other modules
- Low coupling is good as it omits class relationship complexity